4. Tipuri de date

Tipuri de date

- Datele pot fi variabile sau constante
- Fiecare dată are un anumit TIP
- Tipul = mulțime de valori căreia i se poate asocia un nume.
- Valorile sunt constantele de un anumit tip.
- Tipurile pot fi:
  - standard: întreg, real (simplă/dublă precizie), caracter, etc.
  - definite de utilizator.

Modificatori

- Definesc lungimea spațiului de memorie alocate variabilei
  - short
  - long
  - signed
  - unsigned
- În ANSI C:

short int	int	long int
float	double	long double

Tipuri standard

void int float char
long short signed double unsigned

- ✓ char, signed char, unsigned char
- ✓ int, signed int, unsigned int
- ✓ short int, signed short int, unsigned short int
- ✓ long int, signed long int, unsigned long int
- ✓ float
- ✓ double
- ✓ long double

- ✓ Toate declarațiile care încep cu *int* vor fi implicit şi *signed*.
- ✓ Dacă este prezent orice alt modificator de tip, atunci *int* poate lipsi, pentru că este subînțeles.

4.1. Tipul întreg

- ✓ Orice tip întreg este o submulțime a mulțimii numerelor întregi.

- ✓ **int**

{ int Count; … Count = 5; }

Tip şi modificator	Domeniul de valori	Reprezentare în memorie
int short int	[-32768, 32767]	2 octeți cu semn
unsigned int	[0, 65535]	2 octeți fără semn
long int	[-2.147.438.648, 2.147.438.647]	4 octeți cu semn
unsigned long int	[0,4 294 967 295]	4 octeți fără semn

Tipul întreg

- Constante standard asociate tipului întreg
  - INT_MAX (valoarea maximă a tipului ‘int’): 32767
  - INT_MIN (valoarea minimă a tipului ‘int’): -32768
  - LONG_MAX (valoarea maximă a tipului ‘long int’): 2 147 483 647
  - LONG_MIN (valoarea minimă a tipului ‘long int’): -2 147 483 648
  - SHRT_MAX (valoarea maximă a tipului ‘short int’): 32767
  - SHRT_MIN (valoarea minimă a tipului ‘short int’): -32768
  - UINT_MAX (valoarea maximă a tipului ‘unsigned int’): 65535
  - ULONG_MAX(valoarea maximă a tipului ‘unsigned long int’):
  - 4 294 967 295
  - USHRT_MAX (valoarea maximă a tipului ‘int’): 65535.
  - (limits.h)

Exemple pentru unsigned int şi int

Exemplul 1

```
int main ()     {
unsigned int varsta,zi;
…
varsta=20;
 zi=6;
…
return 0;
}
```

Exemplul 2

```
int main ()     {
int masa,forta,acceleratia;
...
masa=300;
acceleratia= -10;
forta=masa*acceleratia;
...
return 0;
 }
```

Tipul întreg - operații

- relații (operatorii relaționali: >, >=, <, >=, ==, !=)

  0 considerat fals, orice altceva diferit de 0 este adevărat
- operații aritmetice (+, -, *, /, %)
- operații logice
  - pe bit (bitwise or |, bitwise and &, bitwise xor ^)
  - pentru variabile (and &&, or | |)

Tipul întreg - operații

AND: 3 & 7

3 = 00000011

7 = 00000111

00000111

00000011

OR: 3 | 7 =7

<< deplasare la stânga

>> deplasare la dreapta

Tipul întreg - operații

- Incrementare x++; ++x
- x - > x+1 (după folosire, respectiv înainte de folosire)
- Decrementare x--; --x
- x - > x-1

- Funcții standard cu argument şi rezultat întreg: abs(x);
- abs (x) = |x| **PC_8.C**

- **PC_9.C**

4.2. Tipul real

- Numerele reale sunt exprimate în virgulă mobilă (*floating point format*).

- În TC există 3 variante de tipuri reale.

Tip şi modificator	Domeniul de valori	Reprezentare în memorie
float	**3.4E-38…3.4E38**	32 biți
double	**1.7E-308…1.7E308**	64 biți
long double	**3.4E-4932…1.1E4932**	80 biți

Tipul real - operații

- relații: se aplică operatorii relaționali clasici, iar rezultatul este o valoare întreagă – pe care o interpretăm adevărată sau falsă
- (>, >=, <, >=, ==, !=)
- 17.4 < 21.3 – *true*
- operații aritmetice: cu operanzi reali şi rezultat de tip real.
- Operatori : + , - , * , /

5/2 cu rezultatul 2, iar 5/2.0 cu rezultatul 2.5.

Tipul real

Observație:

Rezultatele operațiilor asupra întregilor în calculator sunt exacte, spre deosebire de cele asupra numerelor reale care sunt în general aproximative din cauza modului lor de reprezentare limitat (datorită erorilor de rotunjire).

Tipul real - funcții standard

- ✓ abs(x);

  (stdlib.h)

  - abs (x) = |x|
- ✓ pow(x,y); pow10(x);

  (math.h)

  - $\mathbf{x^y}$ $\mathbf{10^p}$
- ✓ sqrt(x); sin(x), cos(x), tan(x)…; x în radiani
- ✓ log(x), log10(x), exp(x);

double - double

Tipul real - funcții standard

ceil(x);

- ✓ returnează valoarea argumentului x rotunjit la cel mai apropiat întreg în sus

floor(x);

- ✓ returnează valoarea argumentului x rotunjit la cel mai apropiat întreg în jos

floor(16.3) = 16; ceil(16.3) = 17;

floor (-1.71) = -2; ceil(-1.71) = -1;

floor(-3.4)=-4; ceil(-3.4)=-3;

floor(0.5) = 0; ceil (0.5)= 1;

floor(-0.5) = -1; ceil (-0.5) = 0;

Tipul real - funcții standard

double modf(double x, double *ipart);

- ✓ returnează partea fracționară a numărului x şi memorează partea întreagă în **ipart**.

PC_x10.C

double fmod (double x, double y)

- ✓ calculează x modulo y, restul împărțirii lui x la y

$x = ay + f; 0<f<y$

Tipul real - funcții standard

- double **poly** (double x, int grad, double coef[]);
- Generează un polinom în x cu gradul precizat în cel de-al doilea argument (‘grad’) şi coeficienții în tabelul ‘coef’. Returnează valoarea polinomului pentru valoarea argumentului dată în variabila x.

rez=poly(4.0,3,val);

double val[] = {2, 3.0, -2.3, 7.0}

p(x)=7 x^3 – 2.3 x^2 +3 x +2 cu arg x=4

rez = 7*4^3 - 2.3*4^2 + 3*4 + 2 =

- Nu e ANSI C -

Generarea numerelor aleatoare

- ✓ int rand (void)

Generează un număr pseduo-aleator în domeniul 0..RAND_MAX

- ✓ void srand(unsigned seed)

Prin apelul acestei funcții cu valoarea argumentului 1, generatorul de numere aleatoare este reinițializat.

stdlib.h
time.h

Generarea numerelor aleatoare

stdlib.h
time.h

- ✓ int random (int num)

returnează un număr aleator între 0 şi num-1.

- ✓ void randomise (void)

Prin apelul acestei funcții generatorul de numere aleatoare este inițializat cu o valoare aleatoare.

Nu sunt ANSI C

4.3. Tipul caracter

- ✓ Definit prin cuvântul standard **char**
- ✓ mulțimea valorilor este reprezentată de mulțimea caracterelor posibile
- ✓ se reprezintă pe un octet
- ✓ pentru caracter cu semn (implicit sau folosind modificatorul *signed*) domeniul de variație este

-128…127, iar pentru caracter fără semn (*unsigned*) domeniul de variație este 0…255 (în ambele cazuri 256 valori).

```
car1 = 'A';
car2 = 'a';
car3 = '\n';     /* newline */
car4 = '\t';     /* tab */
unsigned car_x;
```

Tipul caracter

- ✓ În C mulțimea caracterelor este o submulțime a numerelor întregi, de aici şi dilema dacă o variabilă care conține un caracter trebuie definită *int* sau *char*.

- ✓ Dilema poate să apară, spre exemplu, în cazul în care folosiți funcția *getchar()* – funcție din biblioteca C, stdio.h, care returnează caracterul citit după ce l-a convertit în întreg fără semn.
- ✓ Dacă valoarea este caracter+eof atunci variabila va fi declarată *int*, dacă valoarea citită nu va fi niciodată *eof*, atunci variabila se declară *char*.

Tipul caracter

- ✓ 'x' constantă caracter şi "x" un şir care conține un singur caracter - nu sunt acelaşi lucru.
  - primul este un caracter, folosit pentru a produce valoarea numerică a caracterului x din setul de caracterele al calculatorului;
  - al doilea este un şir de caractere care conține un singur caracter (litera x) şi un \0.

Tipul caracter – operații și funcții standard

- ✓ Se pot aplica operatorii și apela funcțiile standard ca și pentru *int*.

- ✓ Relații - între operanzi de tip caracter

‘C’<‘D’ este adevărat, ‘8’<‘6’ este fals.

PC_x11.C

PC_x12

4.4. Tipul logic

- ✓ există ca tip standard în unele limbaje de nivel înalt, **nu** şi în C, dar se poate defini foarte uşor;
- ✓ Valorile logice (în C, ca rezultat al unei comparații) sunt:
  - fals (zero) şi
  - adevărat (orice valoare diferită de zero).

Tipul logic

- ✓ una dintre variantele cele mai eficiente (timp, memorie) este următoarea:

```
typedef int boolean
   # define TRUE      (1)
   # define FALSE     (0)
```

Tipul logic - operații

- ✓ operatori relaționali: = ,< ,>,<=,>= ,<>

Rezultatul aplicării operatorilor relaționali asupra operanzilor este o valoare logică.

- ✓ operații logice: not - negație (!)

or - disjuncție (||)

and - conjuncție (&&)

Tipul logic - operații

a	b	not a !a	a or b a || b	a and b a && b	
false	false	true	false	false	
false	true	true	true	false	
true	false	false	true	false	
true	true	false	true	true	

Tipul logic

(end-of-file)

eof funcție din io.h care indică atingerea sfârşitului unui fişier şi care are valoarea 1 dacă s-a atins sfârşitul fişierului şi 0 în caz contrar. În caz de eroare valoarea este -1.

EOF (stdio.h) este o constantă pe care o returnează multe din funcțiile din biblioteca C standard atunci când se ajunge la sfârşitul unui fişier. Această valoare este utilă pentru testarea unei condiții.

PC_x13.C **PC_x14.C**